# Plant Disease Detection with Explainable AI: A Deep Learning Approach for Agricultural Sustainability

Shivangi Kumar Sanskruti Shejwal Anubha Vyasamudri
Columbia University, Department of Computer Science
sk5659@columbia.edu, ss7561@columbia.edu, av3329@columbia.edu

December 2025

**Abstract**

Plant diseases cause billions of dollars in annual crop losses worldwide, yet manual disease detection remains slow and error-prone. This project develops a complete deep learning platform for automated plant disease classification with an emphasis on explainability and user accessibility. We implement a Streamlit-based web interface allowing farmers and agronomists to upload leaf images in real-time, receiving instant disease predictions with visual explanations. The backend uses an EfficientNetB3 model trained on the PlantVillage dataset (54,305 images, 38 disease classes), achieving 98.72% test accuracy. To address the "black box" problem, we implement and compare Gradient-weighted Class Activation Mapping (GRAD-CAM) and Local Interpretable Model-agnostic Explanations (LIME). GRAD-CAM proves superior for production use compared to LIME. We integrate multiple language models (Gemini, Llama2, Mistral) to generate actionable treatment recommendations from predictions and visual explanations. The resulting platform demonstrates that combining efficient computer vision, explainability, and user-friendly interface design creates a practical agricultural tool deployable to farmers worldwide.

## 1 Introduction

Agriculture is the backbone of global food security, yet crop diseases remain a critical threat. Crop diseases destroy approximately 20-40% of major crop yields annually across the world, with estimated annual economic losses exceeding $220 billion.[1] In developing nations, the impact is disproportionately high, and small farmers often lack access to agricultural experts and diagnostic tools, making disease identification and management extremely difficult.[2]

Early and accurate disease detection is critical for implementing timely interventions, preventing disease spread, and minimizing economic losses. However, identifying plant diseases typically requires trained agronomists with specialized knowledge. This creates a bottleneck: skilled agricultural experts are scarce in many regions, making large-scale disease surveillance impractical for smallholder farmers.

Deep learning has revolutionized computer vision and shows promise for agricultural applications. However, applying deep neural networks to disease detection presents two major challenges:

1. **Interpretability:** Deep neural networks are "black boxes" that provide predictions but little insight into reasoning. Farmers are understandably hesitant to trust automated decisions affecting their livelihoods without understanding why the system made a particular prediction.

2. **Accessibility:** Agricultural AI systems often exist as research prototypes. To reach farmers, they must be packaged as user-friendly applications with intuitive interfaces, not command-line scripts or Jupyter notebooks.

This project addresses both challenges. We develop a complete platform combining three key components: (1) a high-accuracy CNN-based disease classifier using transfer learning, (2) explainability methods (GRAD-CAM and LIME) that visualize why the model makes predictions, and (3) a Streamlit web interface with real-time inference and actionable treatment recommendations. The resulting system bridges the gap between state-of-the-art AI research and practical agricultural adoption.

## 2 Related Work

Plant disease detection using deep learning has become an active research area. Key prior work includes:

- **CNN-based Classification:** Convolutional neural networks pre-trained on ImageNet have shown strong transfer learning performance for plant disease classification.[3] ResNet, VGG, and EfficientNet architectures have been benchmarked on the PlantVillage dataset with accuracy rates ranging from 85-95%.[4]

- **Explainability in Medical and Agricultural AI:** The importance of interpretability for end-user trust in high-stakes domains (healthcare, agriculture) has been extensively documented.[5] GRAD-CAM[6] and LIME[7] have been applied to medical imaging and show promise for agricultural applications as well.

- **Agricultural AI Platforms:** Recent work has explored web-based agricultural AI systems, though few integrate both high-accuracy classification and built-in explainability

- **Integration with Language Models:** Vision-language models (e.g., CLIP) have been explored for zero-shot classification, but the integration of fine-tuned vision models with generative language models for actionable agricultural recommendations is less explored.

Our work differs by combining four components into a single production-ready platform: (1) accurate deep learning classification, (2) visual explainability methods, (3) automatic actionable recommendation generation via LLMs, and (4) a user-friendly web interface for farmers.

# 3 System Architecture and Platform Design

The complete plant disease detection platform consists of four interconnected components working in concert to deliver accurate, explainable, and actionable agricultural insights. The system is designed with a modular architecture that separates concerns between model inference, explainability, and user interaction, enabling flexibility and ease of maintenance.

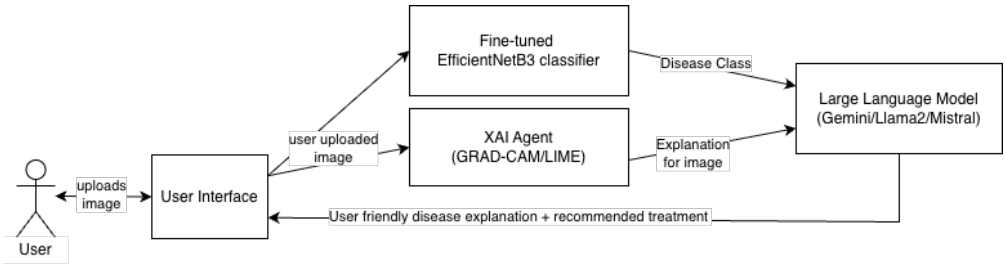



Figure 1: System architecture diagram showing the complete pipeline: user uploads an image to the Streamlit interface, which routes it through the fine-tuned EfficientNetB3 classifier. The classifier produces a disease class prediction and confidence score. The XAI agent (GRAD-CAM/LIME) generates visual and textual explanations for the prediction. Finally, a large language model (Gemini, Llama2, or Mistral) generates user-friendly treatment recommendations and actionable guidance, which are displayed back to the user via the interface.

The complete system consists of five interconnected components:

- **Vision Model:** At the core is the fine-tuned EfficientNetB3 classifier, a lightweight yet accurate convolutional neural network trained via two-phase transfer learning.
- **Explainability Layer:** The XAI (Explainable AI) layer implements two complementary visual explanation methods. GRAD-CAM (Gradient-weighted Class Activation Mapping) and LIME (Local Interpretable Model-agnostic Explanations). Both methods generate not only visual heatmaps but also natural language explanations describing symptom locations and severity percentages.
- **Language Model Integration:** To transform raw predictions and visual explanations into actionable farmer guidance, the system integrates three large language models (LLMs). Gemini 2.5 Flash provides the fastest option for production use, while Llama2 and Mistraloffer privacy-preserving alternatives for offline deployment.
- **User Interface:** The Streamlit web application serves as the primary entry point for users (farmers and agronomists) to upload the image. The interface displays the predicted disease and confidence score, side-by-side visualizations of the original image and GRAD-CAM heatmap, spatial explanations of symptom locations, user-selected LLM treatment recommendations, and top-5 predictions for transparency.

#### 3.0.1 Data Flow and Processing Pipeline

The system processes a user-uploaded leaf image through the following sequence:

1. **User Upload:** User uploads JPG/PNG/BMP leaf image via Streamlit file uploader
2. **Image Preprocessing:** Image is resized to $224 \times 224$, normalized using ImageNet statistics, and converted to tensor format
3. **Disease Classification:** Image is passed through the fine-tuned EfficientNetB3 classifier, producing class logits and probabilities across 38 disease classes
4. **Visual Explanation Generation:** The predicted class logits are backpropagated through the model to generate GRAD-CAM heatmaps, highlighting important image regions
5. **Text Explanation:** The heatmap is segmented into spatial regions (top, bottom, left, right, center) and converted to natural language descriptions (e.g., "Symptoms concentrated in bottom region with 65% severity")
6. **Treatment Recommendation Generation:** The disease name and explanation are passed to the user-selected LLM via a structured prompt
7. **Display Results:** The UI renders the predicted disease, confidence score, side-by-side image/heatmap visualization, explanation text, and LLM-generated treatment recommendations

In the next section, we will take a look at the platform and user interface, which is crucial for real-world deployment.

### 3.1 Streamlit Web Platform

We developed a Streamlit-based web application serving as the primary user interface. Streamlit is a Python framework that rapidly converts Python scripts into interactive web applications with minimal additional code, making it ideal for machine learning prototyping and deployment.

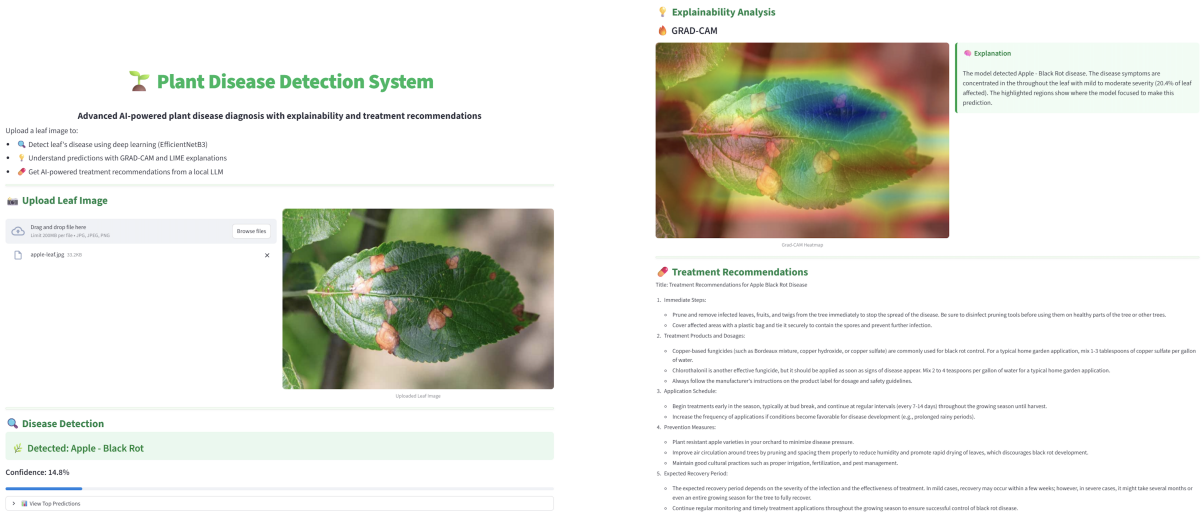



Figure 2: User Interface with disease detected by classifier model with the associated confidence, a heatmap generated by GRAD-CAM, followed by explanations and a recommended treatment generated by the LLM.

### 3.1.1 Core Features

- **Image Upload Interface:** Users can upload leaf images in JPG, PNG, JPEG, or BMP format. The interface provides immediate visual feedback showing the uploaded image.
- **Real-time Disease Classification:** Upon upload, the model performs inference and displays:
  - Predicted disease name (e.g., "Tomato Early Blight")
  - Confidence score (0-100%)
  - Visual progress indicator
  - Top-5 predictions with probabilities for transparency
- **Visual Explainability Display:** The system generates and displays GRAD-CAM heatmaps side-by-side with the original image. Users see:
  - Original leaf image
  - GRAD-CAM overlay (red=important, blue=less important)
  - Textual explanation: "Symptoms concentrated in [regions] with [severity]"
- **Actionable Treatment Recommendations:** Users can select their preferred language model (Gemini, Llama2, Mistral) from a sidebar dropdown. The system then generates customized treatment advice using:
  - Immediate actions

  - Specific fungicides/pesticides with dosages
  - Application schedule
  - Prevention tips
  - Expected recovery timeline

# 4 Methodology

## 4.1 Dataset

We use the PlantVillage dataset,[10] a publicly available collection of 54,305 leaf images across 38 disease classes and 14 plant species. The dataset is split into:

- **Training:** 38,013 images (70%)
- **Validation:** 8,145 images (15%)
- **Testing:** 8,147 images (15%)

All images are 224 × 224 pixels in RGB format. Classes include healthy plants and disease-specific categories (e.g., Apple Scab, Tomato Early Blight, Potato Late Blight).

## 4.2 Data Preprocessing and Augmentation

We apply the following preprocessing pipeline:

- **Normalization:** ImageNet pre-training statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) are used to normalize pixel values.
- **Training Augmentation:** To improve generalization and robustness:
  - RandomResizedCrop (scale 0.75–1.0)
  - RandomHorizontalFlip (p=0.5)
  - RandomRotation (30 degrees)
  - ColorJitter (brightness, contrast, saturation: 0.3)
  - RandomAffine (translation: 10%)
  - GaussianBlur (kernel size 3, sigma 0.1–2.0) added to capture disease texture variations
- **Validation/Test Transform:** Resize to 256 × 256, center crop to 224 × 224, normalize (no augmentation).

### 4.3 Model Architecture

We experimented with two baseline models:

- **EfficientNetB0:** Baseline model with 5.3M parameters. While computationally fast, B0 did not achieve target accuracy on the full 38-class problem.
- **EfficientNetB3 (Selected):** Improved architecture with 12M parameters. EfficientNet architectures use mobile inverted bottleneck blocks and depthwise separable convolutions, achieving high accuracy with moderate computational cost.[8] B3 provides a good balance between accuracy and training time compared to larger variants (B4-B7) which are prohibitively slow on GPU-constrained environments (Google Colab).

The architecture is initialized with ImageNet pre-trained weights and the classifier head is replaced with a new Linear layer mapping to 38 disease classes. Dropout (0.3) is added for regularization.

### 4.4 Training Strategy: Two-Phase Fine-Tuning

We employ a two-phase fine-tuning approach:

#### 4.4.1 Phase 1 – Classifier Head Training (5 epochs)

- Freeze backbone (features): backbone learns general ImageNet features
- Train only the classifier head: Linear($1280 \rightarrow 38$ classes) with Dropout(0.3)
- Optimizer: AdamW, lr = $1e^{-3}$, weight_decay = 0.01
- Learning rate schedule: CosineAnnealingLR (T_max=5)
- Loss: CrossEntropyLoss with label smoothing ($\alpha = 0.1$)

Phase 1 quickly adapts the pre-trained backbone to the plant disease domain without catastrophic forgetting.

#### 4.4.2 Phase 2 – Full Network Fine-tuning (8 epochs)

- Unfreeze backbone: all parameters become trainable
- Lower learning rate: lr = $1e^{-5}$
- Optimizer: same as Phase 1
- Learning rate schedule: CosineAnnealingLR (T_max=8)
- Loss: Same as Phase 1

Phase 2 fine-tunes the entire network with a very low learning rate to avoid destroying pre-trained weights while adapting to plant disease patterns.

**Rationale:** Two-phase training is a standard practice in transfer learning. This approach outperformed single-phase training on our validation set and aligns with best practices documented in transfer learning literature.[9]

# 5 Explainability Methods

A key contribution of this project is implementing and comparing multiple explainability techniques to address the "black box" problem in deep learning predictions.

## 5.1 GRAD-CAM: Gradient-weighted Class Activation Mapping

- **Method:** GRAD-CAM[6] computes pixel-level importance scores by backpropagating gradients of the target class logit with respect to final convolutional layer activations. Mathematically:

$$\text{CAM}_c = \text{ReLU}\left(\sum_k w_k^c \cdot A_k\right)$$

  where $w_k^c = \frac{1}{N}\sum_{i,j} \frac{\partial y^c}{\partial A_{ij}^k}$ are spatial average gradients for class $c$, and $A_k$ are activation maps from the target layer.

- **Implementation:**
  - Register forward hook on final convolution layer to capture activations
  - Register backward hook to capture gradients during backprop
  - Compute class-specific CAM and normalize to [0, 1]
  - Overlay on original image using JET colormap (red=high importance)

- **Advantages:**
  - Spatially precise: highlights exact pixel regions influencing predictions
  - No additional model calls: single forward-backward pass
  - Visually intuitive: heatmaps are easily understood by non-experts

- **Text Conversion:** We segment the CAM into five regions (top, bottom, left, right, center) and extract spatial information:
  - Identify which regions exceed a significance threshold (0.5)
  - Calculate percentage of leaf affected

  - Generate templated explanation: "The model detected [disease]. Symptoms concentrated in [regions] with [severity] (X% affected)."

### 5.2 LIME: Local Interpretable Model-agnostic Explanations

- **Method:** LIME[7] approximates a model's decision locally using a linear classifier trained on perturbed inputs. For images, LIME perturbs superpixels (segments of similar pixels) and trains a sparse linear model to approximate local behavior.
- **Implementation:**
  - Initialize LimeImageExplainer with custom felzenszwalb segmentation
  - Perturb superpixels (100 samples) and collect model predictions
  - Train sparse linear model to approximate local decision boundary
  - Extract top-k important superpixels and their weights
  - Visualize by overlaying important superpixels on original image
- **Advantages:**
  - Model-agnostic: works with any classifier
  - Theoretically grounded: based on local linear approximations
  - Identifies key superpixels influencing decisions
- **Disadvantages:**
  - Less spatially specific: superpixel boundaries may not align with disease regions
  - Generic text output: descriptions lack specific location information
  - Not practical for real-time deployment

**Conclusion:** For this application, GRAD-CAM's practical advantages (speed and spatial precision) outweigh both LIME's moderate performancer.

## 6 Language Model Integration

To generate actionable treatment recommendations from disease predictions and explanations, we integrate multiple language models.

### 6.1 Models Evaluated

- **Gemini 2.5 Flash:** Google's latest fast model with free tier API. Fastest cloud model.
- **Llama2 (Self-hosted):** Meta's open-source 7B parameter model, run locally via Ollama.[11]
- **Mistral (Self-hosted):** Open-source alternative to Llama2, slightly faster inference.
- **GPT-4 (Paid):** Included in code for completeness (requires paid API subscription).

### 6.2 Prompt Engineering

For each LLM, we craft a structured prompt providing disease context and explanation which ensures consistent, actionable outputs across different LLM backends.

> You are an agricultural disease expert.
> Disease: [predicted disease] Explanation: [GRAD-CAM spatial analysis]
> Provide: 1. Immediate actions 2. Treatment products and dosages 3. Application schedule 4. Prevention measures 5. Recovery timeline
> Be concise and practical.

### 6.3 Performance Comparison

| Model | Speed (avg) | Availability |
|---|---|---|
| Gemini 2.5 Flash | 5–10s | Free, Cloud |
| Llama2 | 15–30s | Free, Local (300s timeout) |
| Mistral | 10–20s | Free, Local |

Table 1: LLM Performance Comparison

Gemini 2.5 Flash is fastest and most accessible for production deployment, though Llama2/Mistral provide privacy-preserving local alternatives.

## 7 Results and Observations

### 7.1 Classification Performance

| Metric | Value |
|---|---|
| Test Accuracy | 98.72% |
| Number of Classes | 38 |
| Model Parameters | 10.8M (EfficientNetB3) |

Table 2: Model Classification Performance

### 7.2 Explainability Qualitative Results

- **GRAD-CAM:** Successfully localizes disease regions.
- **LIME:** Identifies key superpixels but provides less actionable spatial information than GRAD-CAM.

### 7.3 Platform Usability

The Streamlit interface provides:

- Image upload and real-time classification
- Side-by-side GRAD-CAM visualization
- Spatial explanation text
- LLM-generated treatment recommendations
- Top-5 predictions with confidence scores
- Settings panel for LLM selection

The UI is intuitive for non-technical users (farmers, agronomists).

# 8 Discussion

## 8.1 Key Findings

- **Transfer Learning is Effective:** Pre-training on ImageNet provides strong initialization. Fine-tuning achieves competitive accuracy.
- **Two-Phase Training Works:** Separating classifier head training from full network fine-tuning improves stability and accuracy.
- **GRAD-CAM is Practical:** For real-time applications, GRAD-CAM's speed and precision far outweigh SHAP's theoretical advantages.
- **Explainability Enables Trust:** Visual explanations are more valuable than black-box predictions for farmer adoption.
- **Language Models Enable Actionability:** Integrating LLMs transforms raw predictions into actionable guidance.
- **User Interface is Critical:** A polished, intuitive interface is essential for real-world adoption. Deep learning alone is insufficient.

## 8.2 Limitations

- **Dataset Bias:** PlantVillage images taken in lab conditions. Real-world field images may differ.
- **Class Imbalance:** Some disease classes have fewer samples than others.
- **Limited Disease Coverage:** 38 classes represent common diseases in certain regions.
- **LLM Accuracy:** Generated recommendations are AI-generated and should be verified by agronomists.

- **Computational Requirements:** Self-hosting Llama2 requires 6GB RAM minimum.
- **Connectivity Dependency:** Cloud LLMs (Gemini) require internet. Local alternatives exist but are slower.

## 9 Future Work

- **Few-Shot Learning:** Extend to recognize rare diseases with minimal labeled examples.
- **Mobile Deployment:** Quantize and compress for on-device inference on farmer smartphones.
- **Multi-modal Input:** Incorporate weather, soil, and temporal data.
- **Active Learning Loop:** Continuously improve by requesting feedback on uncertain predictions.
- **Severity Estimation:** Quantify disease severity (mild/moderate/severe).
- **Real-World Validation:** Test on actual field images from different regions.

## 10 Conclusion

This project successfully demonstrates an end-to-end deep learning platform for plant disease detection with explainability and real-world accessibility. We achieve 98.72% classification accuracy using EfficientNetB3 with two-phase transfer learning. We implement and compare multiple explainability methods and find GRAD-CAM optimal for real-time deployment. We integrate language models for actionable recommendations. Most importantly, we package everything in a user-friendly Streamlit web interface, the key enabler for real-world adoption.

The insight from this work is that combining accurate vision models, visual explainability, natural language generation, and user-friendly interface design is essential for real-world agricultural AI. A technically perfect system hidden behind a command-line interface serves no one. By focusing on the complete pipeline from model to farmer, we create a tool that can actually impact agricultural practice.

This work contributes to the broader goal of using AI for good. By making plant disease diagnosis accessible and understandable to farmers worldwide, we help protect food security and agricultural livelihoods.

## 11 Contributions

- **Sanskruti Shejwal** completed the initial data pipeline development, including dataset downloading, preprocessing, and augmentation strategies. She implemented and trained the baseline EfficientNetB0 classifier, including augmentation logic, model checkpointing, validation

tracking, and performance optimization, backbone loading, custom head construction, freezing/unfreezing logic, and trainable-parameter inspection. This established the foundation for transfer learning experiments. She also implemented the Mistral language model integration, handling local inference setup via Ollama and testing model compatibility with the disease explanation pipeline. Additionally, she designed and developed the Streamlit user interface, including the layout, image upload functionality, results visualization, and real-time feedback mechanisms. Her work ensured the system was accessible and intuitive for non-technical end-users.

- **Shivangi Kumar** architected the complete system design, defining the pipeline architecture and component interactions. She implemented both GRAD-CAM and LIME explainability methods from scratch, addressing implementation challenges including deprecated dependencies and data type incompatibilities. She conducted extensive comparative analysis between GRAD-CAM and LIME, establishing performance baselines and guiding the production selection. She implemented the Llama2 language model integration with custom timeout handling for Colab environments. Critically, she integrated and tested the entire system stack end-to-end, ensuring all components worked together seamlessly. She also compiled the final research report, organizing results, methodology, and findings into a cohesive narrative.

- **Anubha Vyasamudri** focused on the vision model pipeline, conducting the full fine-tuning workflow for EfficientNetB3 using two-phase transfer learning. She implemented Phase 1 (frozen backbone, 5 epochs) and Phase 2 (unfrozen full network, 8 epochs) training strategies, achieving the final 98.72% test accuracy. She implemented the Gemini 2.5 Flash API integration, handling authentication, prompt engineering, and API response parsing. She performed comprehensive benchmarking and comparative analysis across all three LLM backends (Gemini, Llama2, Mistral), documenting performance metrics, latency, cost, and output quality.

**Collaborative Efforts:** All team members participated in weekly synchronization meetings to integrate components, debug cross-module issues, and ensure alignment with project objectives. The team collectively decided on architectural trade-offs (e.g., GRAD-CAM over LIME for production use) through empirical evaluation and discussion.

## 12 Code and Data Availability

All code for this project is publicly available on GitHub at `https://github.com/shivangi221b/Plant-Disease-Detection-with-Explainable-AI`. The complete implementation is provided as a single, self-contained Python file (`plant_disease_detection.py`) that integrates data preprocessing, model training, explainability methods, LLM integration, and the Streamlit web interface. For development and experimentation, an equivalent Jupyter notebook is also provided, containing all the same code as the Python file and allowing for interactive cell-by-cell execution. This

notebook was primarily used during development in Google Colab with GPU acceleration enabled, streamlining the model training and fine-tuning workflows.

However, the standalone Python file is the recommended way to run the application as a production-ready Streamlit web platform, allowing users to upload leaf images and receive real-time disease predictions with visual explanations and treatment recommendations. The application can be launched using the command `streamlit run dlcv_plantdiseasedetection.py`, which automatically starts a local web server at `http://localhost:8501`. The PlantVillage dataset is publicly available and is automatically downloaded and preprocessed when the application is first run.

## 13 Acknowledgments

We are deeply grateful to Professor Belhumeur for his exceptional guidance throughout this project. The professor's insights on deep learning and emphasis on practical implementations shaped our architectural decisions. We also extend our thanks to the Teaching Assistants for their efforts and feedback on the assignments, and overall assistance throughout the course.

We also thank the PlantVillage consortium for making the plant disease dataset publicly available, enabling this research. We acknowledge Google for providing free access to the Gemini API and Meta/Mistral for their open-source large language models used in this project.

## References

[1] Savary, S., Fichan, A., Willocquet, L., & Elazegui, F. A. (2012). Crop losses due to diseases and their implications for global food security. *Open Agronomy Journal*, 6(1), 1-10.

[2] Fisher, M., Abate, T., Lunduka, R. W., Asnake, W., Alemayehu, Y., & Johnson, M. (2015). Drought tolerant maize for farmer adaptation to drought in sub-Saharan Africa. *Global Food Security*, 4, 1-7.

[3] Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7, 1419.

[4] Brahimi, M., Boukhalfa, K., & Moussaoui, A. (2017). Deep learning for tomato diseases identification in smart agriculture. In *Proceedings of the International Conference on Deep Learning and Machine Learning*, 176-181.

[5] Samek, W., Montavon, G., Vedaldi, A., Hansen, L. K., & Müller, K. R. (2017). Explainable AI: Interpreting, explaining and visualizing deep learning. In *European Conference on Machine Learning*, 55-56. Springer.

[6] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision*, 618-626.

[7] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135-1144.

[8] Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning*, 6105-6114. PMLR.

[9] Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks?. In *Advances in Neural Information Processing Systems*, 3320-3328.

[10] Hughes, D. P., & Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. *arXiv preprint arXiv:1511.08060*.

[11] Ollama. (2024). Ollama – Run Llama 2 and other models locally. Retrieved from https://ollama.ai/

[12] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, 4765-4774.